

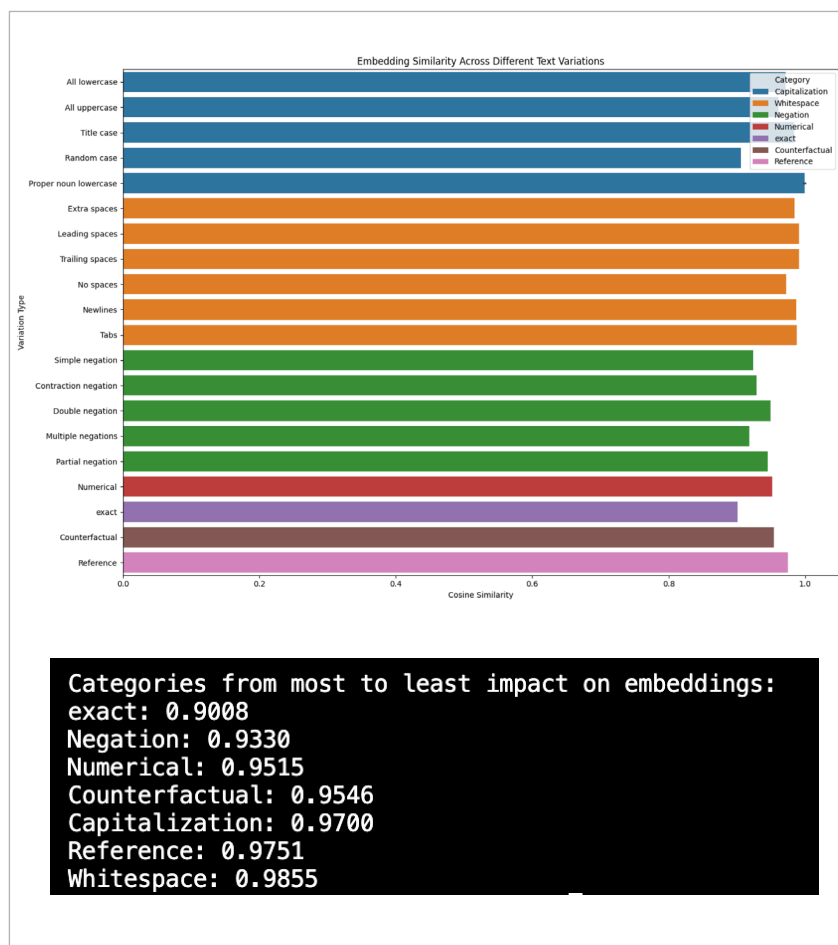


Figure 3: open-ai-text-embedding-3-large embedding score across different test cases

costing \$120 or \$150. Would customers with fixed budgets appreciate seeing products they explicitly can't afford? I wouldn't want to waste time browsing things I can't buy.

Think about cases where ranges versus exact values are crucial — pricing decisions, medication dosing ranges, legal deadlines, safety tolerances, and performance benchmarks. When your model treats 'at least 25 days' and 'exactly 20 days' as basically identical, you've lost critical meaning.

Figures 1, 2 and 3 compare the outputs of msmarco-distilbert-base-tas-b, all-mpnet-base-v2, and open-ai-text-embedding-3-large. You will notice that there is no significant difference between the embedding

scores of these models.

How to work with embeddings anyway

Look, embeddings are still amazing despite these problems. I'm not saying don't use them — I'm saying use them with your eyes open. Here's my battle-tested advice after dozens of projects.

Test your model on real user language before deployment: Not academic benchmarks or sanitised test cases — actual examples of how your users talk. I built patterns that simulate

common variations like negations, typos, and numerical differences. Every system we test fails somewhere — the question is whether those failures matter for your specific application.

Build guardrails around critical blind spots: Different applications have different can't-fail requirements. For healthcare, it's typically negation and entity precision. For finance, it's numbers and temporal relationships. For legal, it's conditions and obligations. Identify what absolutely can't go wrong in your domain and add specialised safeguards.

Layer different techniques instead of betting everything on embeddings: Our most successful systems combine embedding-based retrieval with keyword verification, explicit rule checks, and specialised classifiers for critical distinctions. This redundancy isn't inefficient; it's essential.

Be transparent with users about what the system can and can't do reliably: We added confidence scores that explicitly flag when a result may involve negation, numerical comparison, or other potential weak points. Users appreciate honesty, and it builds trust.

Here's the most important thing I've learned: these models don't understand language the way humans do — they understand statistical patterns. When I stopped expecting human-like understanding and started treating them as sophisticated pattern-matching tools with specific blind spots, my systems got better. Much better.

The blind spots I've described aren't going away soon -- they're baked into how these models work. But if you know they're there, you can design around them. Sometimes, acknowledging a limitation is the first step towards overcoming it. **END** 🐧

By Ritesh Modi

The author is an accomplished software product development professional with extensive expertise in AI/ML, software engineering and management. Currently he works as a principal AI engineer with Microsoft.